Digital VLSI Design

Assignment 1

4-bit Unsigned Array Multiplier from RTL to GDSII

Open-source flow on SkyWater SKY130 (`sky130_fd_sc_hd`)

Icarus Verilog • Yosys • OpenSTA • LibreLane

Team member	**Roll number**
Bhargav	2023112014
Nitin	2023112026

Summary of results	
RTL and gate-level simulation	256 / 256 vectors passed
Synthesis	63 cells (16 flip-flops), 740.71 μm^2
Pre-layout $F_{\max}$ (typical corner)	384.02 MHz ($T_{\min}$ = 2.604 ns)
Final `CLOCK_PERIOD`	8.0 ns after 10 LibreLane runs
Post-route setup slack / $F_{\max}$	+0.400 ns / 131.58 MHz
Signoff	0 DRC, 0 LVS, 0 antenna violations

October 2026

Contents

1 Overview and Environment **2**
1.1 Tools and environment . . . 2
1.2 Repository layout . . . 2

2 Part A: RTL Design and Verification **3**
2.1 Full adder . . . 3
2.2 Array multiplier architecture . . . 3
2.3 Block diagram . . . 4
2.4 Testbench . . . 5
2.5 Simulation results . . . 5
2.6 Hand-worked example (task 5) . . . 5
2.7 Waveform . . . 6

3 Part B: Logic Synthesis with Yosys **7**
3.1 Synthesis script . . . 7
3.2 Part B results . . . 7
3.3 Full cell breakdown . . . 8
3.4 Gate-level (netlist) simulation . . . 8

4 Part C: Static Timing Analysis with OpenSTA **9**
4.1 STA script and constraints . . . 9
4.2 Finding the maximum frequency . . . 9
4.3 Part C results . . . 10
4.4 Critical path . . . 10
4.5 Comparison with the $2N = 8$ full-adder estimate . . . 10

5 Part D: Physical Design with LibreLane **11**
5.1 Configuration and flow . . . 11
5.2 Timing closure: every attempt . . . 11
5.3 Part D results . . . 12
5.4 Timing at every signoff corner . . . 13
5.5 Cells added by place and route . . . 14
5.6 Final layout . . . 15

6 Answers to the Report Questions **16**

1 Overview and Environment

We designed a 4-bit × 4-bit unsigned array multiplier in Verilog, verified it with an exhaustive self-checking testbench, synthesised it to SKY130 standard cells with Yosys, analysed its timing with OpenSTA, and took it through place and route to a GDSII layout with LibreLane. The design registers its inputs and its output, so every timing path in the core runs from one flip-flop to another. The latency is 2 clock cycles and the throughput is one product per cycle.

1.1 Tools and environment

Stage	Tool	Where it ran
RTL and gate-level simulation	Icarus Verilog (`iverilog -g2012`, `vvp`)	pixi environment on the Ubuntu 22.04 host
Waveform viewing	GTKWave	
Logic synthesis	Yosys 0.69+ with ABC	
Static timing analysis	OpenSTA 3.1.0	IIC-OSIC-TOOLS Docker container `iic-osic`, project at `/foss/designs/mult4_team07`
RTL-to-GDSII flow	LibreLane v3.1.0.dev3 (Classic flow: Yosys, OpenROAD, Magic, KLayout, Netgen)	
Layout viewing	KLayout	sky130A layer properties (`report/partD/sky130A.lyp`)
PDK and library	sky130A, `sky130_fd_sc_hd`; Liberty file `sky130_fd_sc_hd__tt_025C_1v80.lib` (typical corner) for Parts B and C	

Table 1: Tools used in each stage of the flow.

1.2 Repository layout

```
README.md               team, environment, every command in order
rtl/full_adder.v        full adder (assign statements)
rtl/mult_array.v        array multiplier: 12 full_adder instances, 16 flip-flops
tb/tb_mult.v            exhaustive self-checking testbench (256 vectors)
synth/synth.tcl         Yosys script     -> mult_array_netlist.v, mult_array_stat.rpt
sta/sta.tcl             OpenSTA script   -> mult_array_timing.rpt
pnr/config.json         LibreLane config -> metrics.json, mult_array.gds (final run)
report/report.pdf       this report
report/partA..partD/    logs, helper scripts, screenshots and tables per part
```

2 Part A: RTL Design and Verification

2.1 Full adder

`rtl/full_adder.v` defines module `full_adder` with inputs `a`, `b`, `cin` and outputs `sum`, `cout`, written only with continuous `assign` statements:

$$sum = a \oplus b \oplus c_{in} \qquad \text{(1 when an odd number of inputs are 1)}$$
$$cout = a\,b + a\,c_{in} + b\,c_{in} \qquad \text{(majority: 1 when two or more inputs are 1)}$$

2.2 Array multiplier architecture

For every bit b_i of B there is one partial product $pp_i = A \wedge b_i$, so $pp_i[j] = a[j] \cdot b[i]$. That gives $4 \times 4 = \mathbf{16\ AND\ gates}$. Row 0 needs no adders. Each of rows 1 to 3 is a 4-bit ripple-carry adder made from 4 full adders, giving $3 \times 4 = \mathbf{12\ full\ adders}$. Each row adds its partial product to the 4 bits passed down from the row above. The lowest sum bit of each row is a finished product bit. Table 2 gives the exact connections.

Row	**Inputs to full adder j ($j = 0 \ldots 3$)**	**Outputs**
0	no adders (AND gates only)	$p[0] = pp_0[0]$; passes down $r_0 = \{0, pp_0[3], pp_0[2], pp_0[1]\}$
1	$pp_1[j]$, $r_0[j]$, $c_1[j]$ $(c_1[0] = 0)$	sum $s_1[j]$, carry $c_1[j{+}1]$; $p[1] = s_1[0]$; $r_1 = \{c_1[4], s_1[3], s_1[2], s_1[1]\}$
2	$pp_2[j]$, $r_1[j]$, $c_2[j]$ $(c_2[0] = 0)$	sum $s_2[j]$, carry $c_2[j{+}1]$; $p[2] = s_2[0]$; $r_2 = \{c_2[4], s_2[3], s_2[2], s_2[1]\}$
3	$pp_3[j]$, $r_2[j]$, $c_3[j]$ $(c_3[0] = 0)$	sum $s_3[j]$, carry $c_3[j{+}1]$; $p[3] = s_3[0]$; $p[7{:}4] = \{c_3[4], s_3[3], s_3[2], s_3[1]\}$

Table 2: Connections of the array (from Section 2 of the assignment).

Implementation in `rtl/mult_array.v`:

- **Ports:** `clk`, `rst_n` (asynchronous, active low), `a[3:0]`, `b[3:0]`, `p[7:0]` (registered). These names match the specification exactly.
- **Input registers:** `a_r` and `b_r` (8 flip-flops) capture `a` and `b` on every rising edge. Reset clears them to 0.
- **Partial products:** `pp`i `= a_r & {4{b_r[`i`]}}`, which gives the 16 AND gates.
- **Adder rows:** three `generate` loops, each with 4 `full_adder` instances (`row1.fa`, `row2.fa`, `row3.fa`), so 12 instances in total. The carry chains `c1`, `c2`, `c3` are 5 bits wide, with bit 0 tied to 0.
- **Output register:** `prod = {c3[4], s3[3:0], s2[0], s1[0], pp0[0]}` is registered into `p` (8 flip-flops). That makes **16 flip-flops** in total.
- The `*` operator is not used anywhere in the RTL.

2.3 Block diagram

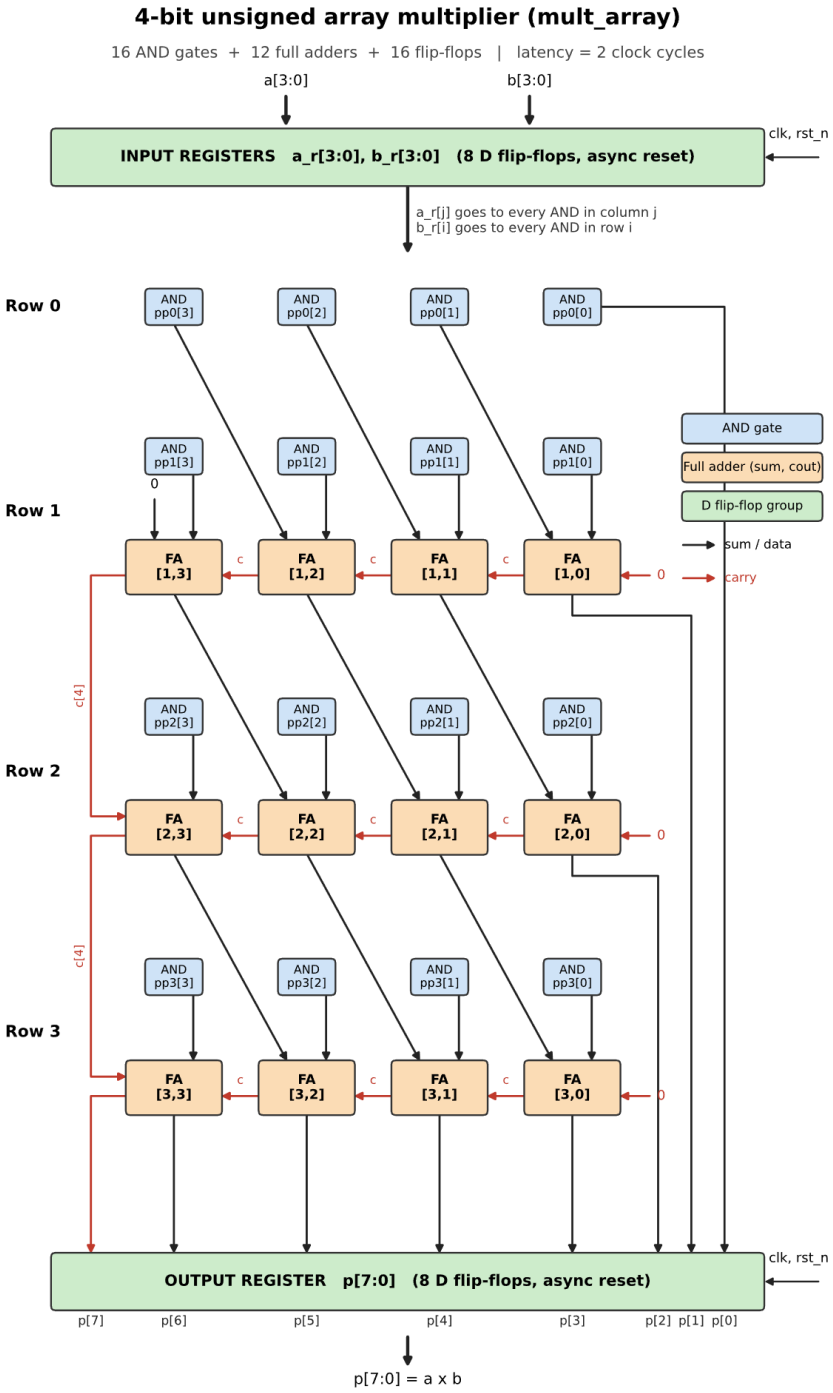


Figure 1: Block diagram of `mult_array`: 8 input flip-flops, 16 AND gates, 12 full adders in three ripple-carry rows, and 8 output flip-flops.

2.4 Testbench

Requirement	How `tb_mult` meets it
10 ns clock; `rst_n` low for the first 2 cycles	`always #5 clk = ~clk`. `rst_n` starts at 0 and is released at the falling edge after the second rising edge (20 ns).
All 256 input combinations	Nested loops `i` = 0…15 (a) and `j` = 0…15 (b).
Wait for the 2-cycle latency, compare with `a * b`	Inputs change at a falling edge. The testbench then waits for two rising edges (capture into `a_r`/`b_r`, then into `p`) and checks `p` at the next falling edge against `expected = i * j`. The `*` operator appears only here.
Print every mismatch; one summary line	`MISMATCH: a=.. b=.. expected=.. got=..` for each error, then exactly one `PASS: 256 vectors checked, 0 errors` or `FAIL: 256 vectors checked, N errors`.
Waveform file `mult.vcd`	`$dumpfile("mult.vcd"); $dumpvars(0, tb_mult);`

Table 3: Testbench requirements and how `tb/tb_mult.v` meets them.

Each vector takes 3 clock cycles (30 ns), so the run ends at $20 + 256 \times 30 = 7700$ ns. This matches the simulator's `$finish called at 7700000 (1ps)`.

2.5 Simulation results

```
$ iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
$ vvp sim
PASS: 256 vectors checked, 0 errors
```

Error-injection test (task 6). To check that the testbench really catches errors, we temporarily changed the sum in `full_adder.v` from `a ^ b ^ cin` to `a ^ b`, which ignores the carry-in. We re-ran the testbench and then restored the file:

```
MISMATCH: a=3 b=3 expected=9 got=5
MISMATCH: a=3 b=6 expected=18 got=10
MISMATCH: a=3 b=7 expected=21 got=9
MISMATCH: a=3 b=11 expected=33 got=29
MISMATCH: a=3 b=12 expected=36 got=20
... (90 more MISMATCH lines)
FAIL: 256 vectors checked, 95 errors
```

95 of the 256 products are wrong. A vector can only fail when some full adder receives a carry-in of 1. Many vectors never produce an internal carry, for example any product with $a = 0$, $b = 0$, or an operand that is a power of two, so 161 vectors still pass. So the testbench does detect a broken adder. The change was undone (`git checkout rtl/full_adder.v`) and the testbench prints PASS again.

2.6 Hand-worked example (task 5)

A = last digit of a roll number + 5. We used roll number 2023112014, whose last digit is 4, so $\mathbf{A = 9 = 1001_2}$. $\mathbf{B = 7 = 0111_2}$ (A $\neq$ 7, so B stays 7). The expected product is $9 \times 7 = \mathbf{63 = 0011\,1111_2} = 0$x3F.

Shift-and-add layout (as in Section 2 of the assignment):

```
          1 0 0 1      A = 9
        x 0 1 1 1      B = 7
  ---------------
          1 0 0 1      pp0 = A AND b0 (b0 = 1)
        1 0 0 1        pp1 = A AND b1 (b1 = 1), shifted 1
      1 0 0 1          pp2 = A AND b2 (b2 = 1), shifted 2
    0 0 0 0            pp3 = A AND b3 (b3 = 0), shifted 3
  ---------------
  0 0 1 1 1 1 1 1      P = 63
```

The same product through the adder rows (Table 2):

Row	pp_i	$+ r_{i-1}$	$= \{c, s\}$	Outputs
0	1001	–	–	$p[0] = 1$, $r_0 = 0100$
1	1001	0100	0 1101	$p[1] = 1$, $r_1 = 0110$
2	1001	0110	0 1111	$p[2] = 1$, $r_2 = 0111$
3	0000	0111	0 0111	$p[3] = 1$, $p[7{:}4] = 0011$

Table 4: Row-by-row evaluation of 9×7 in the array.

$P = \{p[7{:}4], p[3], p[2], p[1], p[0]\} = \{0011, 1, 1, 1, 1\} = 0011\,1111_2 = 63$, the same as the shift-and-add result. No full adder produces a carry for this input pair.

2.7 Waveform

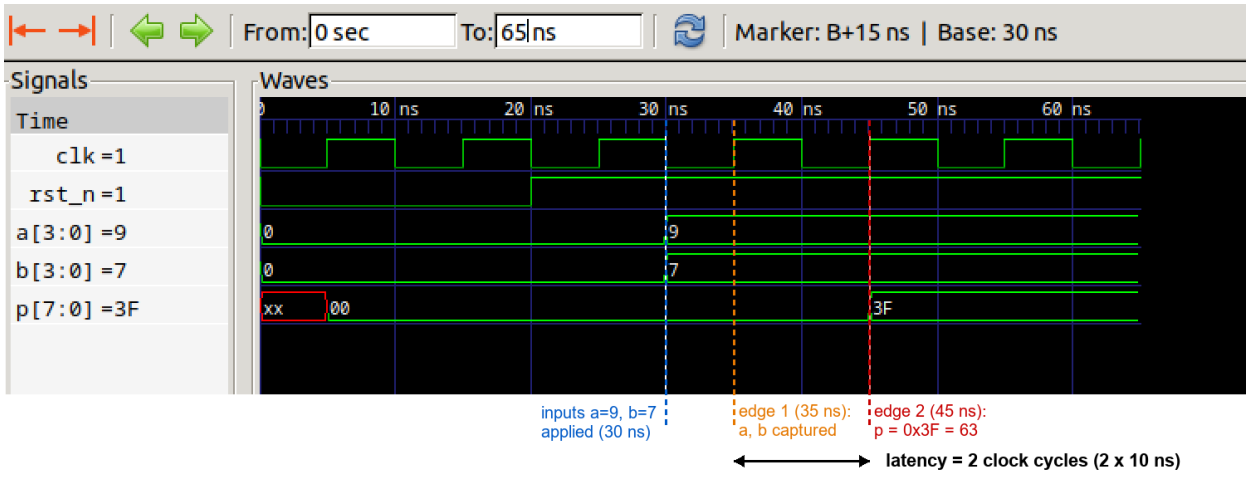


Figure 2: GTKWave view of `clk`, `rst_n`, `a`, `b` and `p` for A = 9, B = 7 (helper testbench `report/partA/tb_example.v`). The inputs change at 30 ns and are captured at edge 1 (35 ns). The product 0x3F = 63 appears on `p` at edge 2 (45 ns), so the latency is 2 clock cycles.

Note. `p` shows `xx` until 5 ns. `rst_n` is declared as `reg rst_n = 0`, so at time 0 there is no falling-edge event to trigger the asynchronous-reset block. The registers are first cleared at the first clock edge, while `rst_n` is still 0. This is a simulation start-up effect only. From 5 ns on, `p` = 00 until the real result arrives. Simulation output of the helper testbench: `a=9 b=7 p=63 (expected 63)`.

3 Part B: Logic Synthesis with Yosys

3.1 Synthesis script

Step	Command in `synth/synth.tcl`	Purpose
1	`yosys read_verilog rtl/full_adder.v rtl/mult_array.v`	Read the design
2	`yosys synth -top mult_array -flatten`	Generic synthesis; flatten the 12 full adders into one module
3	`yosys dfflibmap -liberty $lib`	Map flip-flops to SKY130 cells
4	`yosys abc -liberty $lib`	Map the logic to SKY130 gate cells
5	`yosys opt_clean -purge`	Remove unused wires
6	`yosys tee -o synth/mult_array_stat.rpt stat -liberty $lib`	Cell and area report
7	`yosys write_verilog -noattr synth/mult_array_netlist.v`	Gate-level netlist

Table 5: Steps of `synth/synth.tcl`, run with `yosys -c synth/synth.tcl`.

`$lib` is the typical-corner Liberty file:

```
$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/lib/sky130_fd_sc_hd__tt_025C_1v80.lib
```

3.2 Part B results

Metric	Value
Total number of cells	**63**
Number of flip-flops	**16** (`sky130_fd_sc_hd__dfrtp_1`: D flip-flop with active-low asynchronous reset)
Total cell area (μm^2)	**740.71**
Flip-flop area (μm^2)	**400.38** $= 16 \times 25.024$
Flip-flop area (% of total)	**54.05 %**
The 3 most-used cell types and their counts	`dfrtp_1` $\times$ 16, `nand2_1` $\times$ 8, `a21oi_1` $\times$ 6
Netlist simulation result (PASS/FAIL line)	`PASS: 256 vectors checked, 0 errors`

Table 6: Part B results table (from `synth/mult_array_stat.rpt`).

3.3 Full cell breakdown

Group	Cell (`sky130_fd_sc_hd__...`)	Count	Area (μm^2)
Flip-flops	`dfrtp_1`	16	400.384
Sum logic (XOR/XNOR)	`xnor2_1`	4	35.034
	`xor2_1`	3	26.275
	`xnor3_1`	2	45.043
	`xor3_1`	1	23.773
Carry logic (majority)	`maj3_1`	3	30.029
AND gates	`and2_0`	2	12.512
	`and3_1`	1	6.256
	`and4_1`	1	8.758
	`lpflow_isobufsrc_1` ($A\bar{B}$)	2	12.512
NAND / NOR	`nand2_1`	8	30.029
	`nand2b_1`	1	6.256
	`nand3_1`	3	15.014
	`nand4_1`	1	6.256
	`nor2_1`	1	3.754
	`nor2b_1`	1	6.256
	`nor3_1`	1	5.005
AND-OR-INV / OR-AND-INV	`a21oi_1`	6	30.029
	`a22oi_1`	1	7.507
	`o21ai_0`	1	5.005
	`o311ai_0`	1	8.758
	`o2111ai_1`	1	8.758
Inverters	`clkinv_1`	2	7.507
Total (23 cell types)		**63**	**740.710**

Table 7: All cells in the synthesised netlist, grouped by function.

3.4 Gate-level (netlist) simulation

The synthesised netlist was simulated with the same testbench and the SKY130 cell models, to show that synthesis did not change the behaviour:

```
$ CELLS=$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/verilog
$ iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls \
    tb/tb_mult.v $CELLS/primitives.v $CELLS/sky130_fd_sc_hd.v \
    synth/mult_array_netlist.v
$ vvp gls
```

```
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:49: $finish called at 7700000 (1ps)
```

4 Part C: Static Timing Analysis with OpenSTA

4.1 STA script and constraints

Step	Command in `sta/sta.tcl`	Purpose
1	`read_liberty $lib` (tt_025C_1v80)	Read the cell timing library
2	`read_verilog synth/mult_array_netlist.v; link_design mult_array`	Read and link the netlist
3	`create_clock -name clk -period $T [get_ports clk]`	Clock with period T (default 10 ns, set by `STA_PERIOD`)
4	`set_input_delay 1.0 -clock clk [delete_from_list [all_inputs] [get_ports clk]]`	Input arrival time
5	`set_output_delay 1.0 -clock clk [all_outputs]`	Output required time
6	`report_worst_slack -max`	Worst setup slack
7	`report_checks -path_delay max -digits 3 > sta/mult_array_timing.rpt`	Critical path report

Table 8: Steps of `sta/sta.tcl`, run with `sta -exit sta/sta.tcl`.

The analysis is pre-layout: the clock is ideal (zero skew and insertion delay) and there are no wire parasitics.

4.2 Finding the maximum frequency

```
STA_PERIOD=10.0 sta -exit sta/sta.tcl   -> worst slack max 7.40  (report: +7.396 ns)
T_min = T - WNS = 10.000 - 7.396 = 2.604 ns
F_max = 1000 / T_min = 1000 / 2.604 = 384.02 MHz
STA_PERIOD=2.60 sta -exit sta/sta.tcl   -> worst slack max -0.00 (report: -0.004 ns)
```

At $T = 2.60$ ns ($T_{\min}$ rounded to 0.01 ns) the slack is -0.004 ns, which is effectively zero. The tiny violation comes only from rounding 2.604 ns down to 2.60 ns. This confirms $T_{\min}$.

4.3 Part C results

Metric	Value
Worst slack at $T = 10$ ns (ns)	**+7.396**
$T_{\min}$ (ns)	**2.604**
$F_{\max}$ (MHz)	**384.02**
Slack at $T = T_{\min}$ (ns)	$-$**0.004** at $T = 2.60$ ns (≈ 0)
Critical path start point	`_095_`: input register `a_r[1]` (`dfrtp_1`, CLK→Q)
Critical path end point	`_092_`: output register `p[6]` (`dfrtp_1`, D pin)
Number of cells on the critical path	**10**: 2 flip-flops + 8 combinational cells
Latency (ns) $= 2 \times T_{\min}$	**5.208**
Throughput (million results per second)	**384.02** (one result per clock cycle)

Table 9: Part C results table (typical corner, pre-layout).

4.4 Critical path

#	Instance/pin	Cell	Function in the multiplier	Incr (ns)	Arrival (ns)
1	`_095_/Q`	`dfrtp_1`	input register `a_r[1]`, CLK→Q	0.398	0.398 ↑
2	`_039_/Y`	`clkinv_1`	partial-product AND terms merged with the first adder row	0.074	0.472 ↓
3	`_047_/Y`	`o311ai_0`		0.348	0.820 ↑
4	`_050_/Y`	`a21oi_1`		0.172	0.992 ↓
5	`_063_/X`	`maj3_1`	full-adder carry	0.381	1.373 ↓
6	`_064_/Y`	`xnor2_1`	full-adder sum	0.160	1.534 ↓
7	`_066_/X`	`maj3_1`	full-adder carry	0.385	1.918 ↓
8	`_075_/X`	`maj3_1`	full-adder carry	0.377	2.295 ↓
9	`_079_/X`	`xnor3_1`	final full-adder sum → `prod[6]`	0.175	2.471 ↓
10	`_092_/D`	`dfrtp_1`	output register `p[6]`, setup 0.134 ns	0.000	2.471

Table 10: Critical path from `sta/mult_array_timing.rpt` ($T = 2.60$ ns).

Data arrival = 2.471 ns. Data required = 2.600 − 0.134 = 2.466 ns. Slack = −0.004 ns. At $T = 10$ ns the same path has slack 9.866 − 2.471 = +7.396 ns. The I/O paths are short: input to register is only the 1.0 ns input delay, and register to output is CLK→Q plus the 1.0 ns output delay. So the register-to-register path through the adder array sets $T_{\min}$.

The report also lists the asynchronous reset recovery check (`rst_n` → `_086_/RESET_B`, path group *asynchronous*). Its slack is +9.228 ns at 10 ns and +1.828 ns at 2.60 ns, so it never limits the clock.

4.5 Comparison with the $2N = 8$ full-adder estimate

- The path has **8 combinational cells** and 2.073 ns of logic delay (2.471 − 0.398), about 0.26 ns per cell.
- **5 cells come directly from full adders:** 3 × `maj3` (carry), 1 × `xnor2` and 1 × `xnor3` (sum). The first 3 cells (`clkinv`, `o311ai`, `a21oi`) implement the partial-product ANDs together with the first adder stage. ABC merged these into compound gates.

- The estimate of about $2N = 8$ full-adder delays matches the 8 cells on the path well. In this mapping a full-adder carry is a single `maj3` cell, so one full-adder delay is roughly one cell delay. Because Yosys flattened the adders and ABC re-mapped them, the cells no longer line up one-to-one with `full_adder` instances. This mapping is read from the cell types and the logic around them.

5 Part D: Physical Design with LibreLane

5.1 Configuration and flow

We used LibreLane v3.1.0.dev3 (Classic flow) inside the IIC-OSIC-TOOLS container. The container's default PDK is `ihp-sg13g2`, so every run was started with `--pdk sky130A --scl sky130_fd_sc_hd`. The flow runs:

- synthesis (Yosys)
- floorplanning and the power grid
- tap and decap insertion
- placement, clock-tree synthesis and timing repair
- routing (OpenROAD)
- RC extraction and multi-corner STA
- DRC (Magic and KLayout) and LVS (Netgen)
- GDSII generation

```
{
  "DESIGN_NAME": "mult_array",
  "VERILOG_FILES": ["dir::../rtl/full_adder.v", "dir::../rtl/mult_array.v"],
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 8,
  "FP_SIZING": "absolute",
  "DIE_AREA": [0, 0, 150, 150]
}
```

Listing 1: `pnr/config.json` (final run)

The die is fixed at $150\,\mu\text{m} \times 150\,\mu\text{m}$ (`FP_SIZING = "absolute"`). The core box is $(5.52, 10.88)$ to $(144.44, 138.72)\,\mu$m, which holds 47 rows of `unithd` sites.

5.2 Timing closure: every attempt

Starting period: $1.2 \times T_{\text{min}} = 1.2 \times 2.604 = 3.125$ ns, rounded up to the next 0.5 ns = **3.5 ns**. After each run that failed setup we added 0.5 ns (`report/partD/run_partD.sh 3.5 8.0`). The configuration and metrics of every attempt are kept in `report/partD/attempts/`.

Run	Period (ns)	Setup slack (ns)	Hold slack (ns)	DRC	LVS	Wirelength (μm)	Result
1	3.5	−2.184	+0.114	0	0	2674	FAIL (setup)
2	4.0	−1.684	+0.133	0	0	2674	FAIL (setup)
3	4.5	−1.184	+0.135	0	0	2675	FAIL (setup)
4	5.0	−0.682	+0.135	0	0	2675	FAIL (setup)
5	5.5	−0.208	+0.135	0	0	2688	FAIL (setup)
6	6.0	−0.154	+0.135	0	0	2612	FAIL (setup)
7	6.5	−0.224	+0.136	0	0	2568	FAIL (setup)
8	7.0	−0.137	+0.136	0	0	2560	FAIL (setup)
9	7.5	−0.100	+0.136	0	0	2499	FAIL (setup)
10	**8.0**	**+0.400**	+0.136	0	0	2499	**PASS**

Table 11: All LibreLane runs (slack = worst over all 9 signoff corners).

Between 3.5 ns and 5.0 ns the slack improves by exactly 0.5 ns per step, as expected when only the period changes. From 5.5 ns onwards it improves unevenly and even gets worse at 6.5 ns. This means the critical path delay itself changes from run to run: the netlist and layout are not identical between runs. The most likely reasons are that LibreLane re-synthesises the RTL for each run with the clock period as a timing target, and that the timing-repair and clock-tree steps make different choices for each period. Wirelength also changes between runs (2674 to 2499 μm), which fits this. In the final run the limiting corner is the slow corner `max_ss_100C_1v60` (Table 13).

5.3 Part D results

Metric	Value
CLOCK_PERIOD used (ns)	**8.0**
Number of runs needed	**10** (3.5 → 8.0 ns in 0.5 ns steps)
Core area (μm^2)	**17 759.5** (die area 22 500 = 150 × 150 μm)
Cells after synthesis (from Part B)	**63** (16 flip-flops + 47 logic cells)
Cells after place and route	**1883** instances in total = 86 logic/flip-flop/buffer cells + 245 tap cells + 1552 filler/decap cells (331 excluding filler/decap)
Kinds of cells added	Timing-repair buffers (`buf_2` × 17, `clkdlybuf4s25_1`/`clkdlybuf4s50_1` × 4), clock-tree buffers (`clkbuf_16` × 3), tap cells (`tapvpwrvgnd_1` × 245), decap cells (`decap_*` × 1298), filler cells (`fill_*` × 254)
Wirelength (μm)	**2499** (561 vias)
Setup slack (ns)	**+0.400** (worst corner `max_ss_100C_1v60`)
Post-route $F_{\max}$ (MHz)	**131.58** = 1000/(8.0 − 0.400)
$F_{\max}$ from Part C (MHz)	**384.02**
DRC errors	**0** (Magic 0, KLayout 0, router 0)
LVS errors	**0** (Netgen: 0 device, net, pin or property differences)

Table 12: Part D results table (final run, `CLOCK_PERIOD` = 8.0 ns, from `pnr/metrics.json`).

5.4 Timing at every signoff corner

Corner (process, V, T)	Wire RC	Setup slack (ns)	Hold slack (ns)	Implied $F_{\max}$ (MHz)
tt, 1.80 V, 25 °C	min	4.267	0.360	267.9
	nom	4.240	0.370	265.9
	max	4.209	0.377	263.8
ss, 1.60 V, 100 °C	min	0.500	0.945	133.3
	nom	0.449	0.963	132.4
	max	**0.400**	0.974	**131.6**
ff, 1.95 V, −40 °C	min	5.535	**0.136**	405.8
	nom	5.526	0.143	404.2
	max	5.519	0.147	403.1

Table 13: Setup and hold slack at each of the 9 signoff corners (final run): tt = typical, ss = slow, ff = fast. Implied $F_{\max} = 1000/(8.0 - \text{setup slack})$.

The worst setup slack is register-to-register (`timing__setup_r2r__ws` = 0.3999 ns). Clock skew is about 0.26 to 0.28 ns at every corner. Other signoff numbers:

- 0 antenna violations, 0 power-grid violations, 0 disconnected pins.
- Worst IR drop 0.31 mV.
- Total power at the typical corner: 0.296 mW.
- The flow reports 11 max-slew warnings, all at the slow (ss) corners. These are not setup or hold violations.

5.5 Cells added by place and route

Class	Count	Area (μm^2)	Why the flow adds them
Logic + flip-flops (LibreLane's own synthesis)	62	758.2	16 flip-flops + 42 multi-input cells + 4 inverters. Not added; this is the design itself.
Timing-repair buffers	21	125.1	Fix max slew, capacitance and fanout on long nets. The I/O pins sit on the edge of a 150 μm die, far from the small cluster of logic.
Clock buffers	3	75.1	Clock-tree synthesis: drive the 16 flip-flop clock pins with sharp edges and low skew.
Tap cells	245	306.5	Tie the n-well to VPWR and the substrate to VGND at regular intervals. `sky130_fd_sc_hd` cells have no taps of their own; the taps prevent latch-up.
Decap + filler cells	1552	16 494.6	Decaps put charge reservoirs between VPWR and VGND, which reduces supply noise. Fillers close the gaps in every row so the wells, implants and power rails stay continuous (a DRC requirement). Neither has a logic function.

Table 14: Instance classes in the final layout.

Synthesis gave 63 cells. Place and route has 86 functional cells: LibreLane's own synthesis gives 62 cells, plus 21 repair buffers and 3 clock buffers ($62 + 21 + 3 = 86$). The standard-cell utilisation is only 7.1 % (1264.96 μm^2 of the 17 759.5 μm^2 core). This is why tap, decap and filler cells make up almost all of the instances.

5.6 Final layout

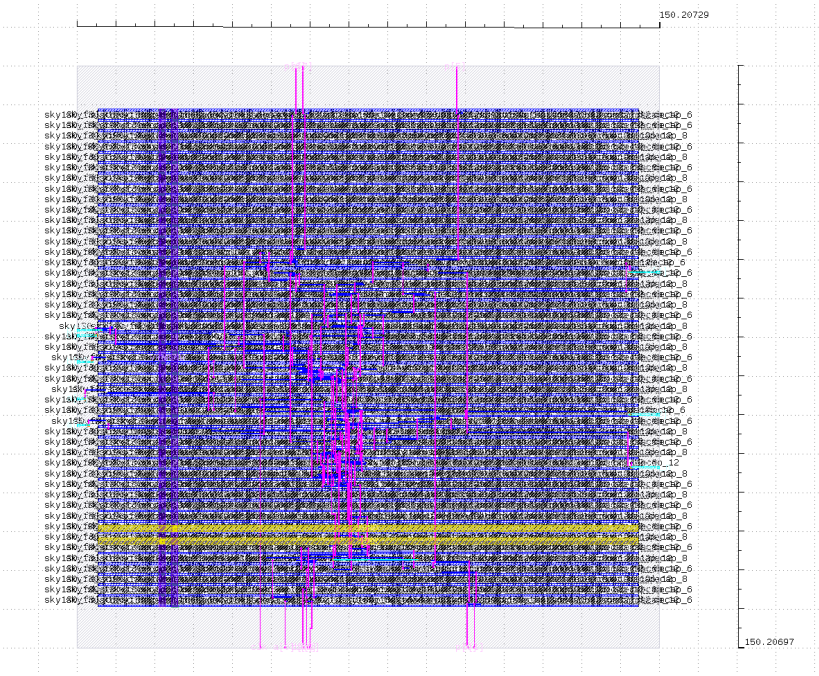


Figure 3: Whole die ($150\,\mu\text{m} \times 150\,\mu\text{m}$) in KLayout. The 47 standard-cell rows are filled with tap, decap and filler cells. The routed logic sits in the centre of the core, and metal wires run out to the I/O pins on the die edges.

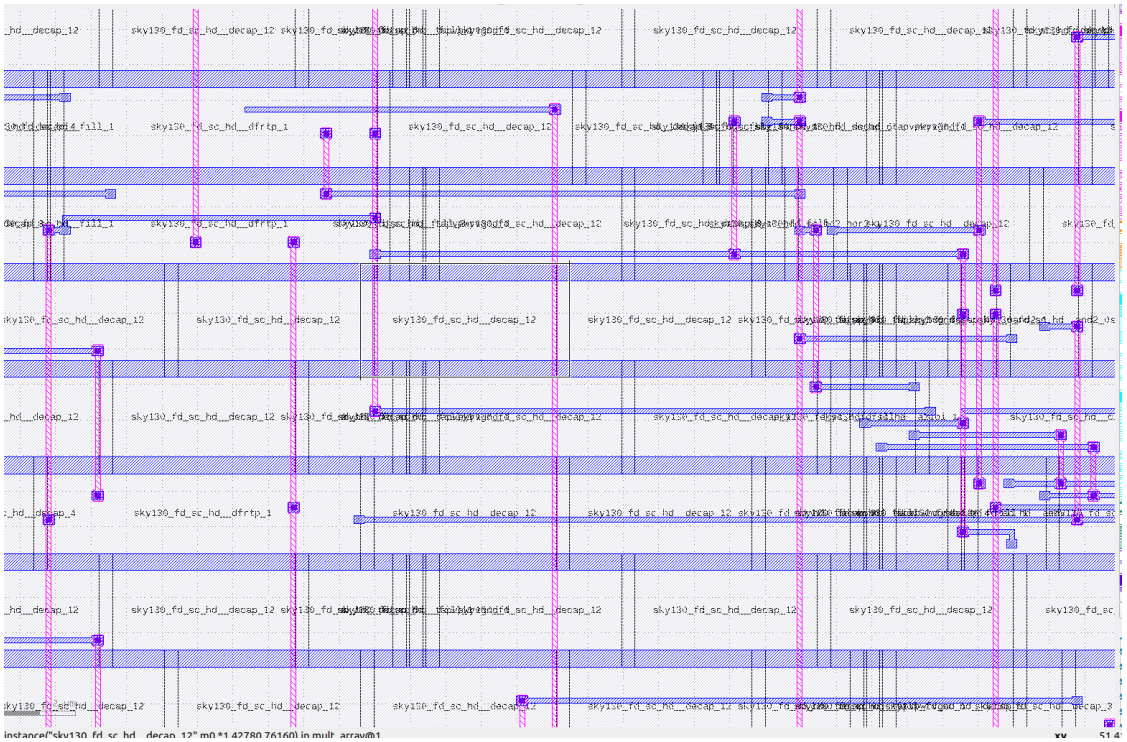


Figure 4: Zoomed view: horizontal power rails (VPWR/VGND) bound each row. The rows hold standard cells (for example `dfrtp_1` flip-flops, `decap_12`, `fill`). Routed metal wires and vias connect the cell pins.

6 Answers to the Report Questions

Q1. The array needs 16 AND gates and 12 full adders. How many cells did synthesis produce for the logic? Explain why the number of cells is not simply 16 + 12.

Synthesis produced 63 cells, of which 16 are flip-flops, so the logic uses **47 cells** (340.3 μm^2) rather than 28. A "full adder" and an "AND gate" are RTL building blocks, not one library cell each. With `-flatten`, Yosys removed the module boundaries, and ABC re-implemented the whole Boolean function with whatever SKY130 cells gave a good area/delay result:

- **One full adder needs at least two cells.** Each full adder becomes a sum cell (`xor2`, `xnor2`, `xor3` or `xnor3`; 10 in total) plus a carry cell (`maj3`) or an equivalent NAND/AOI network. 12 full adders alone would therefore need about 24 cells or more.
- **AND gates are merged into other logic.** Many partial-product ANDs are absorbed into compound gates such as `a21oi`, `a22oi`, `o311ai`, `o2111ai`, `nand3` and `nand4`. Only a few remain as plain `and2/3/4` cells.
- **Constant inputs simplify some adders.** The carry into every row ($c_i[0] = 0$) and the top bit of r_0 are constant 0. Those full adders reduce to half adders, built from an `xor2` plus a `nand2` or `and2`.
- **Inversions need cells.** NAND/NOR/AOI cells are inverting, and a few signals need `clkinv`, `nand2b` or `nor2b`.

The cell count measures how the logic maps onto the library, not how the RTL was written.

Q2. What is the critical path of your design, from which register to which register? How many cells are on it, and how does this compare with the estimate of about 2N full-adder delays?

The critical path starts at input register `a_r[1]` (`_095_`, `dfrtp_1`) and ends at output register `p[6]` (`_092_`, `dfrtp_1`). It has **10 cells**: the launching flip-flop, **8 combinational cells** (`clkinv` → `o311ai` → `a21oi` → `maj3` → `xnor2` → `maj3` → `maj3` → `xnor3`), and the capturing flip-flop. The arrival time is 2.471 ns, giving $T_{\min} = 2.604$ ns. The path follows the carry down through the adder rows into product bit 6, the longest ripple in the array. The estimate for $N = 4$ is $2N = 8$ full-adder delays, which matches the 8 combinational cells well. A full-adder carry maps to a single `maj3` cell, so one full-adder delay is about one cell delay here, around 0.26 ns on average.

Q3. What percentage of the total cell area is flip-flops? Why is this percentage large for such a small design?

Flip-flops take **400.38 of 740.71 μm^2 = 54.05 %** of the cell area, although they are only 16 of the 63 cells (25 %). Each `dfrtp_1` is 25.02 μm^2. That is 6.7 times a `nand2_1` (3.75 μm^2) and about 3.5 times the average logic cell (7.2 μm^2). An edge-triggered flip-flop with asynchronous reset is two latches plus clock buffering and reset logic, which is a few dozen transistors compared with 4 for a NAND2. The other reason is scaling. The array logic grows as N^2 (16 ANDs and 12 adders), while the registers grow only as $4N$ (16 flip-flops). At $N = 4$ the logic is still small, so the registers dominate. For an 8-bit multiplier the flip-flop share would fall to roughly 35 % (see Q6).

Q4. Why is the post-route $F_{\max}$ lower than the $F_{\max}$ from Part C? Which kinds of cells did the flow add, and why is each kind needed?

The post-route $F_{\max}$ is **131.58 MHz**, against **384.02 MHz** from Part C, about 2.9 times lower. The two numbers measure different things:

1. **Corner.** Part C used only the typical library (tt, 1.80 V, 25 °C). LibreLane signs off at 9 corners and reports the worst, which is the slow corner ss, 1.60 V, 100 °C with maximum

wire RC. In the final run the typical corner alone allows $1000/(8.0 - 4.240) \approx$ **266 MHz**, while the slow corner gives 131.6 MHz. The slow corner accounts for about a factor of 2.

2. **Real wires.** Part C had no parasitics. After routing, 2499 μm of wire and 561 vias add extracted resistance and capacitance to every net. The pins are on the edge of a 150 μm die, so some nets are long.
3. **Real clock tree.** Part C used an ideal clock. After clock-tree synthesis there is insertion delay and about 0.26 to 0.28 ns of skew between flip-flops, which reduces the time available for setup.
4. **Different netlist.** LibreLane synthesises the RTL again with its own script and then inserts buffers, so the post-route critical path is not the same 8-cell path analysed in Part C.

The flow added these cells (Table 14):

- **Timing-repair buffers (21):** `buf_2`, `clkdlybuf4s*`. They fix slew, capacitance and fanout limits on long nets.
- **Clock buffers (3):** `clkbuf_16`. They distribute the clock to 16 flip-flops with low skew.
- **Tap cells (245):** they bias the wells and substrate to prevent latch-up.
- **Decap cells (1298):** on-chip supply decoupling against switching noise.
- **Filler cells (254):** keep the wells, implants and power rails continuous across row gaps, so the layout passes DRC.

Q5. What does a DRC check verify, and what does an LVS check verify? Why must both pass before a chip can be manufactured?

DRC (design rule check) verifies the geometry of the layout against the foundry's rules for every layer. It checks minimum widths, spacings, enclosures, overlaps, via rules, density and antenna ratios. These rules encode what the fab can print reliably. A violation can cause shorts, opens or very low yield.

LVS (layout versus schematic) extracts the transistors and their connections from the layout and compares them with the intended netlist: same devices, same nets, same pins. It confirms that the layout is the circuit we designed, with no missing connection, accidental short or swapped pin.

Each check covers what the other cannot. A layout can be perfectly manufacturable (DRC clean) and still implement the wrong circuit, and a layout can be logically correct (LVS clean) but impossible to fabricate. A mask set is very expensive and cannot be fixed after tape-out, so both must pass first. Our final run has **0 DRC errors** in Magic and KLayout (the router's 6 first-iteration violations were fixed in its second iteration) and **0 LVS errors** in Netgen. It also has 0 antenna violations.

Q6. Using the formulas in Section 2, estimate how the number of full adders and the critical path delay would change if the multiplier were 8 bits wide instead of 4. Show your working.

Quantity	Formula	$N = 4$ (ours)	$N = 8$ (estimate)	Change
AND gates	N^2	16	64	$\times 4$
Full adders	$N(N-1)$	12	56	$\times 4.67$
Flip-flops	$2N + 2N$	16	32	$\times 2$
Critical path	$\approx 2N$ full-adder delays	8	16	$\times 2$

Table 15: Scaling from 4 to 8 bits.

Working. In our Part C result the 8 full-adder delays take 2.073 ns of logic, so one full-adder delay is about $2.073/8 \approx 0.26$ ns. The flip-flop overhead is 0.398 (CLK→Q) +0.134 (setup) = 0.532 ns. For $N = 8$:

$$T_{\min}(\text{8-bit}) \approx 0.532 + 16 \times 0.259 = 0.532 + 4.146 \approx 4.68 \text{ ns}$$
$$F_{\max}(\text{8-bit}) \approx 1000/4.68 \approx 214 \text{ MHz} \quad (\text{vs } 384 \text{ MHz for 4 bits; typical corner, pre-layout})$$

So the number of full adders grows about 4.7 times (quadratically in N), while the critical path delay grows only about 2 times in its logic part (linearly in N). $T_{\min}$ rises from 2.60 ns to about 4.7 ns, 1.8 times, because the flip-flop overhead stays the same. For area, if the logic scales with the gate count, $(56 + 64)/(12 + 16) \approx 4.3$ times $340\,\mu\text{m}^2$, it would be about $1460\,\mu\text{m}^2$. With 32 flip-flops at about $800\,\mu\text{m}^2$ the total is about $2260\,\mu\text{m}^2$, and the flip-flop share falls to about 35 %.